

System Overview and User Guide

This application is similar to an online shopping system with two end-users, customers and salespeople. Users have the ability to browse different products available for purchase, create a secure account, search for items, manage their shopping cart, check out, and view past orders. Salespeople are able to update product information, generate weekly sales reports, and view the best-selling items. To facilitate these user and application interactions, the graphical user interface (GUI) was built using PyQt6. The system functionalities were implemented using Python code integrated with SQLite3. Important information such as product and user details was stored in a database, which was accessed at various times throughout the program to check for things like username and password validity, fetch product details based on searched keywords, record new user signups or other updates, and ensure that those data remain persistent and accessible even after the application has finished running or has been rebooted. Below is a comprehensive user guide for a customer and salesperson.

Customer

1. The login screen prompts users to enter their login details or to create an account if they don't already have one. To sign up, the user needs to provide a username, email, and password.
2. After logging in, customers can look up items in the search bar, prompting the system to list all product names and descriptions that are a match.
3. Customers can click on a product to view its details such as the category, price, and stock.
4. If the customer is interested in purchasing the merchandise, they can increment or decrement the quantity, then click "Add To Cart" to proceed to checkout.
5. At checkout, all the products in the user's cart and the total price are displayed. The system stores the prompts the customer for their shipping address and to confirm the transaction.
6. Customers can click the "Account" drop-down menu visible on every screen except for the login page to log out, exit, view past orders, or go to their cart, where they can delete products or change the quantity.

Salesperson

1. A salesperson must already have a predefined account in the system to access the application.
2. After logging in, the salesperson is taken to the main page where they can press the "View/Edit Products" button, after which they will be prompted to search for merchandise by their ID.
3. The system will display the product details, allowing the salesperson to edit the price and stock fields and save those changes.
4. On the main page, the salesperson can also click "View Sales Report" to see the number of distinct orders, products sold, customers with purchases, the average amount spent by customers, and the total profit in the past 7 days.
5. On the main page, the salesperson can press "View Top Selling Products" to see either the top 3 products by total views or the number of distinct orders. The salesperson can pick the option by selecting one of two checkboxes.
6. The salesperson can use the "Account" menu to log out or exit.

Detailed Software Design

We decided to design a GUI instead of a command line interface because the visual components it can offer, such as buttons, search bars, drop-down menus, checkboxes, and input fields, are more intuitive and straightforward to use. The GUI was implemented using PyQt6, and the majority of the functions that display the different visual components and screens are located in `mainGuiManager.py`. Each function in this file facilitates some sort of user action, such as changing to a different screen when a particular button is clicked, and most importantly, calling the Python-integrated SQL functions to access the database and retrieve or update relevant product or user information. These SQL functions are found in `dbManager.py` and each returns a set of values required for the system to properly respond to a user request, such as displaying the product details for a customer to see or updating existing data after a salesperson makes an edit to merchandise. After calling these SQL functions as needed, the GUI-related functions display the data to the user in a clean and organized manner. Below is a table of the SQL functions that cover the customer and user functionalities.

Function	Responsibility	Relationships
<code>check_login</code>	Check if the login credentials exist in the database.	Called by GUI function <code>check_login_info</code> to determine whether to display the salesperson or customer homepage, or the appropriate error message
<code>register_user</code>	Register a new user in the database	Called by GUI function <code>signup_button_clicked</code> when the <code>signup_button</code> is clicked to record the user information into the database
<code>search_product</code>	Search for products by keyword	Called by GUI function <code>customer_search_for_products</code> when a keyword is entered into the search bar
<code>product_details</code>	Get the details of a specific product.	Called by GUI functions <code>view_product_details</code> whenever product details need to be displayed to the user
<code>viewed_product</code>	Adds a product into the <code>viewedProduct</code> table	Called by GUI functions when a specific product is clicked on after a search
<code>add_to_cart</code>	Add a product to the customer's cart	Called by GUI function <code>add_item_to_cart</code> when the add to cart button is pressed
<code>view_cart_items</code>	Get the items in the cart	Called by GUI function <code>view_cart</code> when the View Cart button is clicked

update_product_cart_quantity	Update the quantity of a specific product	Called by GUI function on_quantity_change
remove_from_cart	Remove a product from the customer's cart	Called by the GUI function on_delete_clicked
checkout	Creates a new order with a unique order number. Creates entries in the order_items table for each item in the cart. Updates the stock count in the products table. Empties the cart after the order is created.	Called by the GUI function checkout_button_clicked when the customer presses checkout
view_past_orders	View past orders of the customer	Called by the GUI function open_customer_interface
view_order_details	View the details of a specific past order.	Called by the GUI function view_order_lines
grand_total	Calculate the grand total of a specific order.	Called by the GUI function separate_to_pages
update_product	Allows a salesperson to update a product's price and stock count.	Called by GUI function save_product_changes when the save changes button is clicked.
generate_sales_report	Generate weekly sales report	Called by GUI function view_sales_report_clicked when the view report button is clicked
top_three_products_based_on_order	List the top three products based on the number of distinct orders they were in (includes ties).	Called by GUI function add_top_products_by_distinct_orders when the view top products by orders button is clicked
top_three_products_based_on_views	List the top three products based on the number of views (includes ties).	Called by GUI function add_top_products_by_total_views when the view top products by views button is clicked

Testing Strategy

To test our code, we wrote a script called create_test_database.py which creates the necessary tables and inserts some values into each. Basic testing was done to verify that each user functionality was fulfilled, such as whether registering a new user into the system automatically set their role to “customer” and added the new information collected into the database. After we ensured that all functionalities had been met, we shifted our focus to testing edge cases that may produce intricate errors or unexpected behavior

that could have been overlooked. For example, to ensure that the sales report generated only displays data for the past 7 days, database entries were inserted into tables containing a timestamp column, which was set to the current date, the date one week ago, and one date older than one week ago. Adding these entries allowed the date constraint of the sales report to be tested. We tried to identify any unexpected behavior by coming up with as many invalid actions or inputs as possible, such as not filling out required fields, entering invalid usernames and passwords, and searching for products that don't exist. A bug was uncovered when we tried to search for a nonexistent product ID as a salesperson. The system would display the information of the previously searched product. This bug didn't cause the application to break or raise an exception, since it was a type of logical error. We resolved this issue by forcing the system to display an error message when a product ID doesn't exist in the database.

Teamwork and Coordination

The team's primary form of communication was Discord. We had an initial meeting on October 21st to discuss how we wanted to implement the user interface and divide the work amongst ourselves. We had a meeting at the halfway point on October 29th to report on our progress and resolve any code errors or ambiguities. We organized and tracked our progress by pushing code we have completed to our own branches on GitHub, submitting pull requests to the main branch, and regularly synchronizing changes as we work on our scripts. Below is a table summarizing each member's contributions.

Person	Work Item	Estimated Time Spent	Progress
Emily Lan	Salesperson functionalities	3 hours	Created functions <code>update_product</code> , <code>generate_sales_report</code> , <code>top_three_products_based_on_order</code> , and <code>top_three_products_based_on_views</code>
	Compiling everyone's input into this report	2 hours	Report completed
Vijayasaradhi Poliseti	Creating and handling the performance of UI	5 hours	Created files mainGuiManager.py and designerFile.ui , popupFile.py , <code>GUI.py</code>
Lance Quinto	Customer functionalities	5 hours	Completed all customer functionalities

Additional Features and Extensions

All user functionalities comply with the project specifications. An extension we made was implementing a GUI interface instead of a command line interface because visual elements such as buttons and menus are easier and more intuitive for users to utilize.